LICO: An SIMD-Aware High-Performance Learned Inverted Index Compression Framework (Appendix)

Xianyu Zhu, Qiyu Liu, Guangyi Zhang, Zhibing Sha, Jianwei Liao, Sha Hu and Lei Chen

A Table of Notations

Table I summarizes major notations used in our paper.

Notation	Description
i	The index of key
j	The index of segment
ℓ	The index of block
$\mathcal{K}$	A sorted list
n	The number of keys within $\mathcal{K}$
σ^2	The variance of key gaps within $\mathcal{K}$
$\mathcal{P}_\ell$	The ℓ-th block belonging to $\mathcal{K}$
ϵ	The error bound
$\epsilon_\ell^\star$	The optimal ϵ of block $\mathcal{P}_\ell$
f_ℓ	The $\epsilon_\ell^\star$-PLA fitted on block $\mathcal{P}_\ell$
$f_\ell(i)$	Predicted key for index i
Δ	The residual array
$L(\mathcal{K} \mid \epsilon)$	The number of segments required to satisfy ϵ
b_{seg}	The number of bits to encode one segment
C	A constant used to estimate the number of segments
$S(\epsilon)$	The space cost to encode $\mathcal{K}$ under ϵ
m	The maximum partition number

Table I: Table of notations.

B Proof of Theorem 4.2

Consider the space cost function w.r.t. ϵ:

$$S(\epsilon) = \frac{A}{\epsilon^2} + n \cdot \lceil \log_2(2\epsilon + 1) \rceil + \lceil \log_2 \epsilon \rceil,$$

where constant $A = C \cdot b_{\text{seg}} \cdot n\sigma^2$. To simplify the proof, we consider the continuous relaxation[1] for $\epsilon \in (0, +\infty)$:

$$\widetilde{S}(\epsilon) = \frac{A}{\epsilon^2} + n \cdot \log_2(2\epsilon + 1) + \log_2 \epsilon.$$

The function $\widetilde{S}(\epsilon)$ satisfies $\widetilde{S}(\epsilon) \to +\infty$ as $\epsilon \to 0$ (because $A\epsilon^{-2} \to \infty$) and as $\epsilon \to \infty$ (since $n \log_2(2\epsilon + 1) \to \infty$). Hence, a global minimum exists on $(0, \infty)$. Differentiating and setting the derivative to zero, we have:

$$\widetilde{S}'(\epsilon) = -\frac{2A}{\epsilon^3} + \frac{2n}{(2\epsilon + 1)\ln 2} + \frac{1}{\epsilon \ln 2} = 0$$

$$\iff \frac{2n\epsilon^3}{2\epsilon + 1} + \epsilon^2 = 2A \ln 2.$$

For moderate or large ϵ, $2\epsilon + 1 \approx 2\epsilon$, the critical point can be solved as:

$$\epsilon^\star = \sqrt{\frac{2A \ln 2}{n + 1}} = \sqrt{\frac{2 \ln 2 \cdot C \cdot b_{\text{seg}} \cdot n\sigma^2}{n + 1}} \approx \sqrt{2 \ln 2 \cdot C \cdot b_{\text{seg}} \cdot \sigma^2}.$$

[1]The results for the discrete case remain unchanged as the rounding error is controlled by at most 1.

Algorithm 1 OptimalPartition

Require: List $\mathcal{K}$, partition number m
Ensure: The optimal partition $\mathcal{P}^\star$

```
dp[0][0] ← 0                                          ▷ init DP table
dp[j][ℓ] ← ∞ for all other j, ℓ
split[j][ℓ] ← 0 for all j, ℓ                          ▷ init backtracking table
for j ← 1 to n do
    for ℓ ← 1 to m do
        for i ← ℓ − 1 to j − 1 do
            Compute σ_{i:j} using prefix sums
            ε*_{i:j} ← 4.553 + σ_{i:j}                ▷ Equation (6)
            Cost*_{i:j} ← n_{i:j} · (3.908 + log_2 σ_{i:j})   ▷ Equation (7)
            if dp[i][ℓ − 1] + Cost*_{i:j} < dp[j][ℓ] then
                dp[j][ℓ] ← dp[i][ℓ − 1] + Cost*_{i:j}
                split[j][ℓ] ← (i, ε*_{i:j})           ▷ record the best split
            end if
        end for
    end for
end for
P* ← ∅                                                ▷ backtracking to get partitioning boundaries
j ← n
ℓ ← m
while ℓ > 0 do
    (i, ε*_{i:j}) ← split[j][ℓ]
    Append (D_{i:j}, ε*_{i:j}) to P*
    j ← i
    ℓ ← ℓ − 1
end while
return P*
```

By further checking $\widetilde{S}''(\epsilon^\star) > 0$, we can conclude that $\epsilon^\star$ is the unique global minimum, and the corresponding minimized space cost is:

$$\begin{aligned}\widetilde{S}(\epsilon^\star) &= n + \frac{n+1}{2\ln 2} + \frac{n+1}{2} \log_2 \left(\frac{2A \ln 2}{n+1} \right) \\ &\approx n \cdot \left(1 + \frac{1}{2\ln 2} + \frac{1}{2} \log_2 \left(2 \ln 2 \cdot C \cdot b_{\text{seg}} \cdot \sigma^2 \right) \right) \\ &\approx n \cdot \left(1.957 + \frac{1}{2} \log_2 \left(C \cdot b_{\text{seg}} \cdot \sigma^2 \right) \right)\end{aligned}$$

In practice, as $\epsilon \in \mathbb{N}^+$, the optimal discrete error bound $\epsilon_d^\star$ must appear in $\{\max(1, \lfloor \epsilon^\star \rfloor - 1), \lfloor \epsilon^\star \rfloor, \lceil \epsilon^\star \rceil, \lceil \epsilon^\star \rceil + 1\}$. Accordingly, the exact global minimum $S_d^\star$ satisfies $\widetilde{S}(\epsilon^\star) \le S_d^\star \le \widetilde{S}(\epsilon^\star) + (n + 1)$, meaning that at most one extra bit per key will be introduced.

C Pseudo Code of DP-based Partitioning

The DP-based optimal partitioning algorithm is given in Algorithm 1. Clearly, such a DP algorithm takes $O(mn^2)$ time and $O(mn)$ space.

D Proof of Theorem 4.4

For simplicity, we rewrite the space cost function as a set function:

$$f(X) = |X| \cdot \left(\xi + 0.5 \log_2 \sigma_X^2\right),$$

where $|X|$ constant $\xi = 1.957 + 0.5 \log_2(C \cdot b_{\text{seg}})$. For two disjoint sets A and B, we first show a key property of $f(\cdot)$:

$$\boxed{f(A) + f(B) - f(A \cup B) \leq \frac{n}{2} \log_2(U/L).}$$

Let $n_A = |A|$, $n_B = |B|$, and $n_{A \cup B} = |A \cup B| = n_A + n_B$ (as $A \cap B = \phi$). We then have,

$$\begin{aligned} f(A \cup B) &= n_{A \cup B} \cdot (\xi + 0.5 \log_2 \sigma_{A \cup B}^2), \\ f(A) &= n_A \cdot (\xi + 0.5 \log_2 \sigma_A^2), \\ f(B) &= n_B \cdot (\xi + 0.5 \log_2 \sigma_B^2). \end{aligned}$$

Let $\Delta = f(A) + f(B) - f(A \cup B)$. As the gap variance $\sigma^2 \in [L, U]$, we can upper bound Δ as follows:

$$\begin{aligned} \Delta &= \frac{n_A}{2} \log_2 \left(\frac{\sigma_A^2}{\sigma_{A \cup B}^2} \right) + \frac{n_B}{2} \log_2 \left(\frac{\sigma_B^2}{\sigma_{A \cup B}^2} \right) \\ &\leq \frac{n_A + n_B}{2} \log_2(U/L) \\ &\leq \frac{n}{2} \log_2(U/L) \end{aligned}$$

Let Δ^t denote the total space cost change after the t-th iteration in the greedy algorithm. We then have:

$$\text{ALG} \leq f(\mathcal{K}) + \sum_{t=1}^{m^g - 1} \Delta^t \leq f(\mathcal{K}) + \frac{m^g \cdot n}{2} \log_2(U/L), \qquad (1)$$

where $m^g \leq m$ is the practical number of partitions produced by the greedy algorithm. We then find the upper bound of $f(\mathcal{K})$ and the lower bound of OPT as follows:

$$f(\mathcal{K}) = n \cdot (\xi + 0.5 \log_2 \sigma_{\mathcal{K}}^2) \leq n \cdot (\xi + 0.5 \log_2 U), \text{ and}$$

$$\text{OPT} = \sum_{j=1}^{m} |\mathcal{P}_j^\star| \cdot (\xi + 0.5 \log_2 \sigma_{\mathcal{P}_j^\star}^2) \leq n \cdot \xi + \frac{nm}{2} \log_2 U,$$

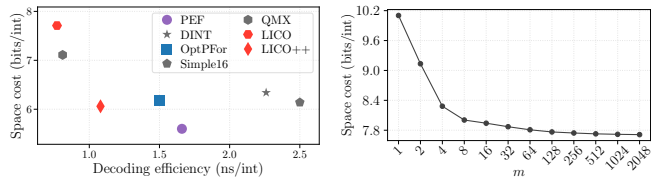
where $\{\mathcal{P}_1^\star, \cdots, \mathcal{P}_m^\star\}$ is the partitioning scheme found by the optimal DP-based partitioning algorithm. By combining these inequalities, when n is sufficiently large, we can derive the approximation ratio of the proposed greedy algorithm as:

$$\begin{aligned} \frac{ALG}{OPT} &\leq \frac{2n\xi + n \log_2 U + m^g n \log_2(U/L)}{2n\xi + mn \log_2(U/L)} \\ &\leq \frac{2\xi + \log_2 U + m^g \log_2(U/L)}{2\xi + m \log_2(U/L)} \\ &\approx \frac{m^g}{m} = O(1). \end{aligned}$$

Thus, we complete the proof. □

E Generalization Beyond Specific Workloads

When applying LICO to new datasets with substantially different gap statistics, a simple and robust procedure is to estimate C on a small probe set and re-estimate it if R^2 falls below a chosen criterion. To empirically evaluate LICO's generalization ability, we construct a DNA dataset from the human reference genome (2.6B integers) by sampling 30% of adenine (A) positions from each primary chromosome. Its gap statistics (mean 12.19, variance 1.78085×10^7) differ substantially from the web datasets. We re-estimate C on this dataset and obtain a robust value $C = 0.05749$ with $R^2 = 0.64$. As shown in Figure Ia, both LICO and LICO++ perform well on DNA; moreover, the partitioning algorithm captures local variance structure effectively (Figure Ib), reducing space usage. These results indicate that the constant C and partition number m are not tuned to a single workload and that the framework transfers to different data distributions.



(a) Space-time trade-off on DNA. **(b) Space cost of LICO w.r.t. m.**

Figure I: Generalization analysis of LICO on DNA.

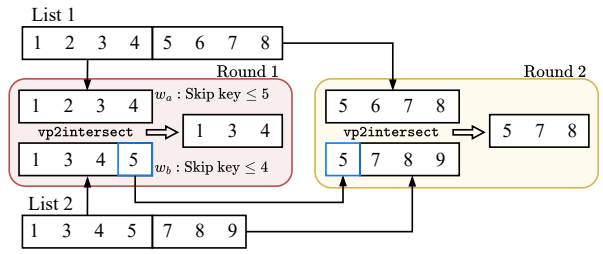


Figure II: An example for the SIMD-based list intersection.

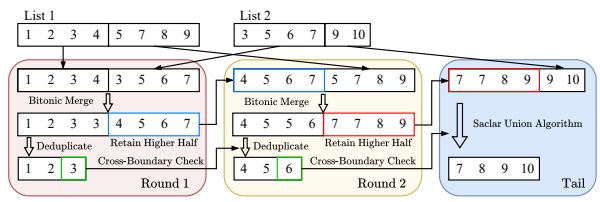


Figure III: An example for the SIMD-based list union.

F Running Examples of List Operations

Figure II presents a running example of the SIMD-based list intersection method. In each round, the algorithm slides two windows w_a and w_b over the input lists and uses one vp2intersect instruction to compute their intersection. Then, each window skips all keys smaller than the maximum key in the other window and proceeds to the next round.

Figure III shows an example of the SIMD-based list union algorithm. In each round, the algorithm merges two fixed-length input blocks into a single sorted sequence. It then deduplicates the lower half of the sequence via predecessor comparison with a cross-boundary check and outputs unique keys into the union result. The upper half is carried forward as one input block for the next round. For the other input block in the next round, the algorithm selects the candidate block whose first element is smaller from either list.